

Topic: Types of Queue - Circular Queue, Priority Queue

Q. Explain the limitations of a simple linear queue. How do a Circular Queue and a Priority Queue overcome specific challenges. Describe their working principles.

[DEF] Definition to Remember

A Circular Queue connects the last array index back to the first, solving the memory wastage problem of linear queues. A Priority Queue dequeues elements based on their assigned priority value rather than strict arrival time (FIFO), allowing critical tasks to be processed first.

Queue Types: Circular & Priority Queues

In a standard linear array queue, once the rear pointer reaches the maximum index ($\text{SIZE} - 1$), no new elements can be enqueued. This happens even if spaces are empty at the front due to earlier dequeue operations, leading to severe memory wastage.

2. Circular Queue

Solution: The queue forms a logical circle. The next position after the last index is index 0.

* Working Principle: Uses the modulo operator (%) to wrap pointers around.

* Ne

```
[0] <-- front
/      \
[3]      [1]
\      /
[2] <-- rear
(If SIZE=4, next rear after 3 is  $(3+1)\%4 = 0$ )
```

3. Priority Queue

Problem: A standard queue strictly processes the oldest element first. Some real-world tasks (like CPU interrupts) are urgent and cannot wait in a FIFO line.

Solution

* Working Principle:

* Ele

[MUST-WRITE] Must-Write Points (for 10 marks)

1. Linear Queue Limitation: Wastes memory; cannot insert when rear is at the end, even if front is empty.
2. Circular Queue: Connects the last index to the first using the modulo operator $(\text{rear} + 1) \% \text{SIZE}$.
3. Circular Queue perfectly reuses freed memory space. It is full when $(\text{rear} + 1) \% \text{SIZE} == \text{front}$.
4. Priority Queue: Elements are removed based on a priority value, not strict FIFO.
5. In Priority Queues, if priorities tie, FIFO determines the order.
6. Priority Queues are optimally implemented using Heaps (Min/Max) for $O(\log n)$ efficiency.
7. Circular Q used in round-robin scheduling; Priority Q used in CPU interrupt handling.

[RECALL] Quick Recall

Linear Q wastes memory -> Circular Q wraps around $((rear+1)\%SIZE)$, reuses space -> Priority Q breaks FIFO for urgent tasks, served by highest priority -> implemented via Heaps

Complete Executable C Code: Circular Queue Array Implementation

```
#include <stdio.h>

#define MAX 5

struct CircularQueue {
    int items[MAX];
    int front;
    int rear;
};

void initQueue(struct CircularQueue *q) {
    q->front = -1;
    q->rear = -1;
}

int isFull(struct CircularQueue *q) {
    return (q->rear + 1) % MAX == q->front;
}

int isEmpty(struct CircularQueue *q) {
    return q->front == -1;
}

void enqueue(struct CircularQueue *q, int val) {
    if (isFull(q)) {
        printf("Circular Queue Full! Cannot enqueue %d\n", val);
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % MAX;
    }
    q->items[q->rear] = val;
    printf("Enqueued: %d\n", val);
}

int dequeue(struct CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Circular Queue Empty! Cannot dequeue\n");
        return -1;
    }
    int val = q->items[q->front];
    if (q->front == q->rear) { // Single element left
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % MAX;
    }
    return val;
}
```

```
void display(struct CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is Empty\n");
        return;
    }
    printf("Circular Queue: ");
    int i = q->front;
    while (1) {
        printf("%d ", q->items[i]);
        if (i == q->rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    struct CircularQueue q;
    initQueue(&q);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);
    display(&q);

    printf("Dequeued: %d\n", dequeue(&q));
    printf("Dequeued: %d\n", dequeue(&q));
    display(&q);

    enqueue(&q, 50);
    enqueue(&q, 60);
    display(&q);

    return 0;
}
```